

JAVA 17 OCP · YDS STUDY SERIES

Unit 12 · Modules — Practice Quiz

Teknik İngilizceyi bağlam içinde öğren · kısa tekrarlarla kalıcılaştır

Bu belge sekiz adet **OCF tarzi özgün çalışma sorusu** içerir; sorular gerçek sınavdan alınmamıştır. Önerilen süre 25–30 dakikadır; istersen 1–4 ve 5–8 olarak iki oturuma böl. Command ve descriptor sorularında önce classpath/module path ayrımını, sonra readability ve access kurallarını değerlendir.

Sorular

Soru 1

Odak: Built-in named module

Aşağıdaki programın çıktısı nedir?

```
public class ModuleIdentity {
    public static void main(String[] args) {
        Module module = String.class.getModule();
        System.out.print(module.isNamed() + ":" + module.getName());
    }
}
```

- A. `false:null`
- B. `true:java.lang`
- C. `true:java.base`
- D. Kod derlenmez.

Soru 2

Odak: `open module` declaration

Aşağıdaki `module-info.java` için hangisi doğrudur?

```
open module zoo.tours {
    opens zoo.tours.api;
}
```

- A. Başarıyla derlenir; yalnız `zoo.tours.api` reflection'a açılır.
- B. Başarıyla derlenir; `opens` satırı gereksiz olsa da yasak değildir.
- C. **Does not compile**; `open module` declaration'ında ayrıca `opens` directive kullanılamaz.
- D. Yalnız `exports zoo.tours.api`; eklenirse derlenir.

Soru 3

Odak: `exports` ve `opens`

Ordinary explicit named module için hangisi doğrudur?

- A. `exports`, package'ı yalnız deep reflection'a açar.
- B. `opens`, başka module'lerin public type'ları normal source code ile import etmesini tek başına sağlar.
- C. `exports`, normal erişimi; `opens`, özellikle deep reflection erişimini yönetir.
- D. Export edilmeyen package içindeki `main()` method command line'dan hiçbir zaman başlatılamaz.

Soru 4

Odak: Named, automatic ve unnamed module

Aşağıdaki ifadelerden **hangi ikisi doğrudur?**

- A. Automatic module module path üzerinde bulunur; diğer resolved named module'ları ve unnamed module'ü okur.
- B. Unnamed module tipik olarak classpath üzerindeki code'u temsil eder.
- C. Ordinary explicit named module, command-line override olmadan unnamed module'ı varsayılan olarak okur.
- D. Automatic module mutlaka source içinde yazılmış bir `module-info.java` içerir.
- E. Unnamed module doğrudan `jlink` image'ına eklenebilir.

Soru 5

Odak: Service directives

Servisi **doğrudan** `ServiceLoader.load(Service.class)` ile **arayan** açık isimli bir consumer modülü ile provider modülü için doğru directive çifti hangisidir? Servis API türlerine gerekli `requires/exports` erişiminin sağlandığını varsay.

- A. Consumer: `provides Service with Consumer`; provider: `uses Service`
- B. Consumer: `exports Service`; provider: `opens Implementation`
- C. Consumer: `uses Service`; provider: `provides Service with ServiceImpl`
- D. Consumer ve provider yalnız `requires java.base` yazmalıdır.

Soru 6

Odak: English → Turkish / YDS

Aşağıdaki cümleyi doğal Türkçeye çevir ve `only if` yapısının mantıksal yönünü açıkla:

A public type is accessible to another named module only if its package is exported to that module and the reading module has the required readability.

Soru 7

Odak: `requires transitive` ve erişim zinciri

Üç açık isimli modülün ilgili dosyaları aşağıdadır. Başka yönerge veya komut satırı erişim istisnası yoktur; kaynaklar module path kullanılarak derlenir. `app` derlemesini başarılı kılan **tek değişiklik** hangisidir?

```
// api/module-info.java
module api { exports api; }
// api/api/Item.java
package api;
public class Item {}

// middle/module-info.java
module middle { requires api; }

// app/module-info.java
module app { requires middle; }
// app/app/Main.java
package app;
public class Main { api.Item item; }
```

- A. `api` tanımlayıcısında `exports api` yerine `opens api` yazmak.
- B. `middle` tanımlayıcısında `requires transitive api` yazmak.
- C. `app.Main.item` alanını `public` yapmak.
- D. Hiçbir değişiklik yapmamak; bütün bağımlılıklar kendiliğinden geçişlidir.

Soru 8

Odak: Ana sınıfı modüler başlatma

Aşağıdaki dosyalar `mods/greeting` dizinine başarıyla derlenmiştir; terminalin geçerli dizini `mods` dizininin bulunduğu üst dizindir.

```
// module-info.java
module greeting {}
// greet/Main.java
package greet;
public class Main {
    public static void main(String[] args) { System.out.print("Hello"); }
}
```

Hangi komut programı **başarıyla başlatıp Hello yazdırır**? Tek seçenek seç.

- A. `java --module-path mods -m greeting/greet.Main`
- B. `java --module-path mods -m greeting.greet.Main`
- C. `java --module-path mods -m greeting/greet/Main`
- D. Hiçbiri; `exports greet;` yoksa başlatıcı `main()` metodunu çağıramaz.

Dil aktarımı: “The package need not be exported for this launch.” cümlesini çevir. `need not` ifadesi izin mi, yasak mı, zorunluluk yokluğu mu bildirir?

Cevaplar ve açıklamalar

Soru 1 — C

- **A yanlış:** `String` classpath'teki unnamed module'a değil built-in named module'a aittir.
- **B yanlış:** `java.lang` package adıdır, module adı değildir.
- **C doğru:** `String`, named `java.base` module'ündedir; çıktı `true:java.base` olur.
- **D yanlış:** `java.lang.Module` Java 9'dan beri API'nin parçasıdır ve kod Java 17'de derlenir.

Soru 2 — C

- **A yanlış:** `open module`, bütün package'ları reflection'a açar; ayrıca `opens` yazılması syntax olarak yasaktır.
- **B yanlış:** Directive yalnız gereksiz değil, compilation error nedenidir.
- **C doğru:** Java 17 compiler bu descriptor'ı **Does not compile** olarak reddeder.
- **D yanlış:** Bir `exports` directive'i eklemek yasak `opens` directive'ini geçerli hâle getirmez.

Soru 3 — C

- **A yanlış:** `exports`, normal compile-time/runtime erişimini düzenler; yalnız reflection directive'i değildir.
- **B yanlış:** `opens` normal import/access için `exports` yerine geçmez.
- **C doğru:** İki directive'in sınavdaki temel ayrımı normal access ve deep reflection'dır.
- **D yanlış:** Export, inter-module access içindir; module launcher export edilmeyen package içindeki uygun `main()` method'u başlatabilir.

Soru 4 — A ve B

- **A doğru:** Automatic module module path'tedir; diğer resolved named module'ları ve unnamed module'ü okur. Bu readability, migration'ı kolaylaştırır.
- **B doğru:** Classpath'teki ordinary code unnamed module içinde çalışır.
- **C yanlış:** Ordinary explicit named module ile unnamed module arasında varsayılan readability edge'i yoktur.
- **D yanlış:** Explicit descriptor içeren artifact explicit named module olur; automatic module adı manifestten veya JAR filename'ından türetilir.
- **E yanlış:** `jlink`, resolved named module graph'ı ile runtime image oluşturur; unnamed module doğrudan bu graph'ın module'ü değildir.

Soru 5 — C

- **A yanlış:** Consumer service'i `uses` ile bildirir; provider implementation'ı `provides ... with ...` ile kaydeder.
- **B yanlış:** `exports` ve `opens`, service discovery directive'lerinin yerine geçmez.
- **C doğru:** Consumer için `uses Service`, provider için `provides Service with ServiceImpl` doğru rol eşleşmesidir.
- **D yanlış:** `java.base` dependency'si implicit olsa da service relationship için gerekli directive'leri sağlamaz.

Soru 6 — Örnek çeviri

“Public bir type, başka bir named module tarafından ancak package'ı o module'e export edilmişse ve okuyan module gerekli readability'ye sahipse erişilebilir.”

`only if`, sonrasındaki koşulları **gerekli koşul** yapar: erişim varsa hem uygun export hem readability bulunmalıdır. Bu yapı “if” ile ters yönde okunmamalıdır. `is exported` passive voice, `reading` ise module'ü niteleyen present participle'dir.

Soru 7 — B

- **B doğru:** `middle, api` bağımlılığını geçişli bildirince `middle` modülünü okuyan `app, api` modülünü de okur. `api` paketi zaten dışa açılmıştır; kaynaklar derlenir.
- **A yanlış:** `opens` normal kaynak kodu erişimini sağlamaz; `app` için okunabilirlik sorunu da sürer.
- **C yanlış:** Alanın erişim düzeyi, kullanılan türün modülünü okunabilir yapmaz.
- **D yanlış:** Normal `requires` bağımlılığı geçişli yayılmaz. İlk sürüm `package api is not visible` nedeniyle derlenmez.

Soru 8 — A

- **A doğru:** Başlatma sözdizimi `modül/paket.AnaSınıf` biçimindedir; çıktı `Hello` olur.
- **B yanlış:** Eğik çizgi olmadığında bütün ifade modül adı gibi yorumlanır; belirtilen modül bulunamaz.
- **C yanlış:** Modül/sınıf ayırıcısından sonraki tam sınıf adı, paketler arasında nokta kullanır.
- **D yanlış:** Modül başlatıcısının uygun `main()` metodunu çalıştırması için paketin dışa açılması gerekmez.

Çeviri: “Bu başlatma için paketin dışa açılması gerekmez.” `need not`, zorunluluk yokluğudur; yasak bildirmez.